Rewriting Binaries for Bug Hunting: Reproducing RetroWrite for Binary-Only Fuzzing and Sanitization

Chirag Suthar (2025MCS2105)
Haleel Sada (2025MCS2741)

SIL7165: Networks & System Security
Semester 2, 2025–26

March 2026

1 Introduction

1.1 Area

Software security is a very important topic in today's world. Most of the software we use daily, like web browsers, media players, and operating systems, is closed-source. This means users only get the compiled binary and not the original source code. Finding security bugs in such closed-source software is a big challenge. The area of *binary analysis and instrumentation* deals with techniques that can analyse and modify compiled programs without needing their source code.

1.2 Problem

Memory corruption bugs, such as buffer overflows, use-after-free, and out-of-bounds reads/writes, are among the most dangerous security vulnerabilities. Tools like AddressSanitizer (ASan) [2] and American Fuzzy Lop (AFL) [3] are very effective at finding these bugs. However, these tools normally require the source code of the program to work properly. For binary-only software, the existing approaches have serious problems. Dynamic Binary Translation (DBT) tools like QEMU [4] and Valgrind [5] can instrument binaries at runtime, but they are very slow – typically 10x to 100x slower than native execution. This slowness makes them impractical for fuzzing, where we need to run millions of test inputs. Existing static binary rewriting tools like Uroboros [8] and ramblr [9] use heuristics to guess which values in the binary are addresses and which are just numbers, and these heuristics are often wrong, leading to broken binaries.

1.3 Solution

RetroWrite [1] solves this problem by taking a clever approach to static binary rewriting. Instead of guessing which values are addresses, RetroWrite uses the *relocation information* that is already present in position-independent code (PIC) binaries. Modern 64-bit Linux binaries are compiled as PIC by default, so this covers most real-world software. RetroWrite reads the binary, uses relocations to figure out exactly which values are addresses, replaces them with symbolic labels (this step is called *symbolization*), and produces a reassembleable assembly file. This assembly file can then be modified to add security checks (like ASan) or coverage tracking (like AFL), and then reassembled into a new working binary. The whole process is done offline (before running the program), so there is no runtime translation overhead.

1.4 Evaluation

The RetroWrite paper shows very strong results. Their AFL instrumentation (called `afl-retrowrite`) achieves 4.2x to 5.6x higher fuzzing throughput compared to QEMU-based AFL on real-world programs. Their binary ASan (`ASan-retrowrite`) is 3x faster than Valgrind memcheck and detects 80% more bugs. On SPEC CPU2006 benchmarks, ASan-retrowrite incurs only about 65% runtime overhead, comparable to source-level ASan's 73% overhead, which is very impressive for a binary-only tool.

1.5 Takeaway

The main takeaway is that for a large and important class of binaries (64-bit PIC), static binary rewriting can be done soundly (without heuristics) by leveraging relocation information. This makes it possible to add source-level security tools like ASan and AFL to closed-source software with near-native performance.

2 Background and Related Work

2.1 Background

Binary Analysis: When software is compiled, the source code is converted into machine code (a binary). Analysing this binary to understand its behaviour is called binary analysis. This is important for security because we often need to check closed-source software for vulnerabilities.

Disassembly: The first step in binary analysis is disassembly – converting machine code back into assembly instructions. There are two main approaches: linear sweep (go through all bytes sequentially) and recursive descent (follow the control flow). Tools like IDA Pro [12], Ghidra, and radare2 [13] do this.

Position-Independent Code (PIC): Modern binaries are compiled as position-independent, meaning they can be loaded at any memory address. This is needed for ASLR (Address Space Layout Randomization) [11], a security feature. PIC binaries contain relocation entries that tell the loader how to fix up addresses. RetroWrite uses these relocations.

AddressSanitizer (ASan): ASan [2] is a memory error detector developed by Google. It uses "shadow memory" – for every 8 bytes of real memory, ASan keeps 1 byte that says whether those bytes are safe to access. Before every memory read/write, ASan checks the shadow memory. If the access is to poisoned memory, ASan reports a bug.

Coverage-Guided Fuzzing: Fuzzing is a testing technique where random inputs are fed to a program to find crashes. Coverage-guided fuzzers like AFL [3] track which parts of the code are executed by each input and prefer inputs that reach new code paths. This is much more effective than blind random fuzzing.

Dynamic Binary Translation (DBT): DBT tools like QEMU translate binary code at runtime, one block at a time. This allows inserting instrumentation but has high overhead (10x–100x slower).

Reassembleable Assembly: The idea is to take a binary and produce assembly code that can be modified and reassembled. The key challenge is *symbolization* – figuring out which values in the binary are addresses (references) and which are plain numbers (scalars). If you get this wrong, the reassembled binary will crash.

2.2 Related Work

Table 1 shows a comparison of existing solutions and their limitations.

Table 1: Comparison of related binary instrumentation approaches

Tool	Type	Sound?	Fast?	Scalable?	Limitations
QEMU [4]	DBT	Yes	No (10–100x)	Yes	Very slow for fuzzing
Valgrind [5]	DBT	Yes	No (2–300x)	Yes	Too slow for fuzzing
Intel PIN [6]	DBT	Yes	No (2–5x)	Yes	Runtime overhead
DynamoRIO [7]	DBT	Yes	No (2–5x)	Yes	Runtime overhead
Uroboros [8]	Static	No	Yes	No	Wrong heuristics
ramblr [9]	Static	No	Yes	Partial	Still uses heuristics
DynInst [10]	Static	Partial	Yes	Partial	Unsound instrumentation
RetroWrite [1]	**Static**	**Yes**	**Yes**	**Yes**	**Only 64-bit PIC**

Dynamic binary translation tools (QEMU, Valgrind, PIN, DynamoRIO) are sound and work on any binary, but they are slow because they translate code at runtime. Static approaches like Uroboros and ramblr try to produce reassembleable assembly, but they use heuristics for symbolization that can be wrong, leading to broken binaries. DynInst can statically instrument binaries but its instrumentation is sometimes unsound. RetroWrite is the first tool that achieves all three goals: soundness, speed, and scalability, by leveraging relocation information in PIC binaries.

3 Problem Statement

3.1 System Model

The system consists of the following entities:

- **Target Binary:** A 64-bit Linux ELF binary compiled as position-independent code (PIC/PIE). This is the closed-source software that we want to test for security bugs. The binary may be a shared library (.so) or an executable. The source code is not available.

- **RetroWrite Framework:** The binary rewriting tool. It takes the target binary as input and produces a reassembleable assembly file. It uses the Capstone disassembly framework and pyelftools for ELF parsing. It is implemented in about 2,000 lines of Python.
- **Instrumentation Passes:** Modules that modify the reassembleable assembly to add security checks. The two main passes are: (i) ASan pass – adds memory safety checks before every memory access, and (ii) AFL pass – adds coverage tracking at every basic block.
- **Assembler and Linker:** Standard tools (GCC/Clang assembler) that compile the modified assembly back into a working binary.
- **Fuzzer (AFL):** The fuzzing tool that runs the instrumented binary with many random inputs to find crashes.
- **Security Analyst:** The person who uses RetroWrite to test closed-source binaries for bugs.

The interaction flow is: Security Analyst provides a target binary to RetroWrite. RetroWrite disassembles it, symbolizes it, and produces assembly. The instrumentation pass adds ASan/AFL checks. The assembler creates a new instrumented binary. AFL then fuzzes this binary to find bugs.

3.2 Threat Model

- **Attacker's Goal:** The attacker wants to exploit memory corruption vulnerabilities (buffer overflows, use-after-free, etc.) in the target binary to achieve arbitrary code execution, data theft, or denial of service.
- **Attacker's Capabilities:** The attacker can provide crafted inputs to the target program. The attacker knows the binary is a 64-bit PIC binary running on Linux with standard protections like ASLR and DEP enabled.
- **Defender's Goal:** The defender (security analyst) wants to find these memory bugs before the attacker does, by using RetroWrite to add ASan checks and AFL fuzzing to the binary.
- **Assumptions:** The binary is compiled as PIC (position-independent code), it is not obfuscated, and it has relocation information intact (not stripped). The binary is for x86-64 architecture.

3.3 Key Challenges

1. **Symbolization Accuracy:** The hardest part of binary rewriting is correctly identifying which values are addresses and which are plain numbers. Getting this wrong means the rewritten binary will crash or behave incorrectly.
2. **ABI Compatibility:** The instrumentation must not break the Application Binary Interface. For example, leaf functions on x86-64 can use 128 bytes below the stack pointer without allocating them (red zone). Pushing data for instrumentation would overwrite this area.
3. **Register Allocation:** Adding instrumentation code needs registers to work with, but all registers may already be in use. RetroWrite uses liveness analysis to find dead registers, but this analysis must be conservative to avoid breaking the program.
4. **Setting Up the Build Environment:** RetroWrite depends on specific versions of Python libraries, Capstone, and compilers. Getting everything to work together was a practical challenge we faced.
5. **Reproducing Paper Results:** The paper's experiments used SPEC CPU2006 benchmarks, which require a paid license. We used alternative benchmarks (bzip2, custom fuzz targets) for our evaluation.

4 RetroWrite Design

RetroWrite proposes a static binary rewriting framework that produces reassembleable assembly from 64-bit PIC binaries. The key insight is that PIC binaries contain relocation information that can be used for sound symbolization – no heuristics needed.

4.1 Overview

RetroWrite works in five steps, as shown in Figure 1:

Figure 1: Overview of the RetroWrite pipeline. The binary goes through five stages to produce an instrumented version.

1. **Preprocessing:** Load the binary's code (.text) and data sections. Read relocations and symbols. Disassemble the code using Capstone. Build a control-flow graph (CFG) by identifying basic blocks and edges.
2. **Symbolization:** This is the core step. RetroWrite processes three types of references:
   - *Control-flow references:* Calls and jumps are converted to use assembler labels.
   - *PC-relative addressing:* Instructions that reference data relative to the instruction pointer are adjusted to use labels.
   - *Data relocations:* Entries in the relocation table point to addresses in the data sections. These are replaced with labels.

   The output is a reassembleable assembly file where all addresses are represented as labels.
3. **Instrumentation Passes:** Security tools are implemented as passes over the assembly. Two passes are provided:
   - *ASan pass:* Before every memory read/write instruction, insert code to check the shadow memory. If the memory is "poisoned" (freed, out of bounds), report an error.
   - *AFL pass:* At every basic block, insert code to update a coverage bitmap so AFL can track which code paths are exercised.
4. **Optimization:** RetroWrite performs register liveness analysis to minimize overhead. Instead of saving all registers before instrumentation, it only saves registers that are actually in use.
5. **Reassembly:** The modified assembly is compiled using a standard assembler (e.g., GCC) to produce the final instrumented binary.

4.2 ASan Instrumentation Details

Table 2 shows how ASan-retrowrite handles different memory regions compared to source-level ASan.

Table 2: Redzone policy comparison: ASan vs ASan-retrowrite

Memory Region	Source ASan	ASan-retrowrite
Heap	Individual object	Individual object
Stack	Individual, 32-byte	Stack-frame, 8-byte
Global	Individual, padded to 64-byte	No redzone

The main difference is that ASan-retrowrite cannot add redzones to global variables because the disassembly cannot recover the semantic boundaries between adjacent globals. For stack variables, it uses frame-level redzones instead of individual variable redzones because the binary does not contain variable boundary information.

5 Completed Evaluation

We reproduced the core functionality of RetroWrite by setting up the tool, running it on test programs and real-world binaries, and comparing the results.

5.1 Experimental Setup

- **Machine:** Ubuntu Linux with x86-64 architecture
- **Tools:** RetroWrite (cloned from GitHub), AFL++, GCC, Clang
- **Target programs:** (i) Demo programs from RetroWrite repo (heap.c with heap overflow and use-after-free bugs), (ii) bzip2 1.0.8 (real-world compression tool), (iii) Custom fuzz target (our own program with planted bugs)

5.2 Experiment 1: ASan Bug Detection

We compiled the heap.c demo program as a normal PIE binary, then used RetroWrite to add ASan instrumentation to the binary.

Table 3: ASan bug detection results on demo program

Bug Type	Original Binary	RetroWrite + ASan
Heap buffer overflow	Not detected (silent)	Detected
Use-after-free	Not detected (silent)	Detected

The original binary ran the buggy code without any error. The RetroWrite-instrumented binary immediately caught both bugs and printed a detailed ASan error report showing exactly where the illegal memory access happened.

5.3 Experiment 2: Real-World Binary Rewriting (bzip2)

We used RetroWrite to rewrite the bzip2 binary (105KB). RetroWrite successfully:

- Disassembled bzip2 into assembly (807KB assembly file)
- Reassembled it into a working binary (101KB)
- The rewritten binary produced **identical output** to the original when compressing and decompressing files
- Also generated an ASan-instrumented version (2.5MB assembly)
  This confirms the paper's claim that RetroWrite can handle real-world binaries soundly.

5.4 Experiment 3: Extended ASan Testing

We wrote our own test program (`asan_test.c`) with four distinct memory bugs to further validate RetroWrite's ASan capabilities beyond the basic demo.

Table 4: Extended ASan detection results on our custom test program

Bug Type	Original Binary	RetroWrite + ASan
Heap buffer overflow	Not detected (silent)	Detected
Use-after-free	Not detected (silent)	Detected
Stack buffer overflow	Not detected (crash)	Detected
Double free	Not detected (crash)	Detected

5.5 Experiment 4: Attack Surface Analysis

We also performed a systematic attack analysis on our fuzz target program (`fuzz_target.c`). We tested 10 different attack types and classified each by whether it was possible, whether the original binary crashed, and whether ASan detected it.

Table 5: Attack surface analysis on fuzz_target.c

Attack Type	Possible?	Original Crashes?	ASan Detects?
Heap buffer overflow	Yes	No (silent corruption)	Yes
Null pointer deref	Yes	Yes (segfault)	Yes
Stack buffer overflow	Yes	Yes (segfault)	Yes
Format string	No	—	—
Use-after-free	No	—	—
Double free	No	—	—
Integer overflow	No	—	—
Command injection	No	—	—
Information leak	No	—	—
Race condition	No	—	—

RetroWrite's ASan instrumentation detected all three possible memory safety attacks, including the heap buffer overflow that silently corrupts memory without crashing – the most dangerous type of bug since the attacker remains undetected.

5.6 Experiment 5: AFL Fuzzing Performance

We wrote a custom fuzz target program with intentional bugs (heap overflow, null pointer dereference, stack overflow). We compiled it as a PIE binary, then used RetroWrite to add AFL instrumentation.

Table 6: Fuzzing throughput comparison

Method	Speed (exec/sec)	Source
Source-level AFL (best possible)	4790	Our measurement
RetroWrite AFL (binary-only)	**4244 (88.6%)**	Our measurement
QEMU AFL	∼800	Reported in paper

RetroWrite achieved 88.6% of source-level AFL speed without needing the source code. This is consistent with the paper's claim of near-native performance. Note that the QEMU AFL figure is from the original paper, as we did not set up QEMU-based fuzzing. The QEMU-based approach is roughly 5x slower.

5.7 Comparison with Existing Solutions

Table 7: Quantitative comparison with existing solutions (overhead figures from the RetroWrite paper [1])

Tool	Needs Source?	Runtime Overhead	Bug Detection
Source ASan	Yes	1.73x	Best
RetroWrite ASan	**No**	**1.65x**	**Good**
Valgrind memcheck	No	20–300x	Lower
QEMU + AFL	No	10–100x	Low throughput
RetroWrite + AFL	**No**	**∼1x**	**Near source-level**

6 Planned Evaluation

For the endterm submission, we plan to extend our work in the following ways:

6.1 Extension 1: Fix rep prefix ASan Instrumentation

The RetroWrite paper explicitly mentions that `rep movsb` and `rep stosb` instructions (used by memcpy and memset) are not instrumented by ASan-retrowrite. This is marked as a TODO in the source code at `rwtools_x64/asan/instrument.py`. We plan to add shadow memory checks for both the start and end addresses of the memory region accessed by these instructions. This will allow ASan-retrowrite to catch more buffer overflows.

6.2 Extension 2: Basic Block Coverage Pass for x64

The RetroWrite repository has an AFL coverage pass for ARM64 but not for x64. We plan to create a standalone coverage pass for x64 that inserts `inc byte [bitmap + BLOCK_ID]` at each basic block. This will be useful for coverage measurement without needing the full AFL setup.

6.3 Extension 3: Stack Canary Insertion

We plan to add a pass that inserts stack canaries into binaries compiled without `-fstack-protector`. At function entry, a random canary value is pushed onto the stack. Before every `ret` instruction, the canary is checked. If it has been overwritten (indicating a stack buffer overflow), the program aborts.

6.4 Evaluation Plan

We will evaluate these extensions by:

- Testing the rep prefix fix on programs that use memcpy/memset with intentional overflows
- Measuring coverage accuracy of our coverage pass compared to source-level coverage
- Testing stack canary insertion on programs with known stack overflow vulnerabilities
- Measuring the performance overhead of each extension

7 Conclusion

In this project, we studied and reproduced the RetroWrite paper, which presents a static binary rewriting framework for adding security instrumentation to closed-source binaries. RetroWrite uses relocation information in 64-bit PIC binaries to perform sound symbolization, avoiding the heuristic-based approaches of prior tools. We successfully set up RetroWrite, tested its ASan instrumentation on both demo and custom programs (detecting heap overflows, use-after-free, stack overflows, and double free bugs), performed a systematic attack surface analysis across 10 attack types, rewrote a real-world binary (bzip2) with correct output, and measured AFL fuzzing throughput at 88.6% of source-level speed. Our results confirm the paper's key claims. For the endterm, we plan to extend RetroWrite by fixing the rep prefix ASan limitation, adding a standalone coverage pass, and implementing stack canary insertion.

References

[1] S. Dinesh, N. Burow, D. Xu, and M. Payer, "RetroWrite: Statically Instrumenting COTS Binaries for Fuzzing and Sanitization," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2020, pp. 1497–1511.

[2] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A Fast Address Sanity Checker," in *Proc. USENIX Annual Technical Conference (ATC)*, 2012.

[3] M. Zalewski, "American Fuzzy Lop (AFL)," 2017. [Online]. Available: `https://lcamtuf.coredump.cx/afl/`

[4] F. Bellard, "QEMU, a Fast and Portable Dynamic Translator," in *Proc. USENIX Annual Technical Conference (ATC)*, 2005.

[5] N. Nethercote and J. Seward, "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation," in *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2007.

[6] C.-K. Luk et al., "Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation," in *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2005.

[7] D. Bruening, T. Garnett, and S. Amarasinghe, "An Infrastructure for Adaptive Dynamic Optimization," in *Proc. International Symposium on Code Generation and Optimization (CGO)*, 2003.

[8] S. Wang, P. Wang, and D. Wu, "Reassembleable Disassembling," in *Proc. USENIX Security Symposium*, 2015.

[9] R. Wang et al., "Ramblr: Making Reassembly Great Again," in *Proc. NDSS Symposium*, 2017.

[10] A. R. Bernat and B. P. Miller, "Anywhere, Any-Time Binary Instrumentation," in *Proc. ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools (PASTE)*, 2011.

[11] PaX Team, "Address Space Layout Randomization (ASLR)," 2003. [Online]. Available: `https://pax.grsecurity.net/`

[12] Hex-Rays, "IDA Pro Disassembler," [Online]. Available: `https://hex-rays.com/ida-pro/`

[13] S. Alvarez, "radare2: UNIX-like reverse engineering framework," [Online]. Available: `https://rada.re/n/radare2.html`